



01211373 Machine Learning and Programming for Industry

From Neural Network Foundations to PyTorch

Lecture 7 — Deep Learning

Part 1 → Part 2

WEEKS 1–6 — CLASSICAL ML

- One interface: fit / predict / score
- Pipelines keep preprocessing honest
- Cross-validation gives you evidence, not a number
- You now have a tuned, defensible baseline (Checkpoint 1)

TODAY

- Multilayer perceptrons: stacking Week 1's neuron
- Backpropagation — the intuition, not the algebra
- PyTorch tensors and autograd
- Dataset, DataLoader, and the training loop
- Re-solve the Week 2 fault task with a neural net

Why Switch Tools Now?



An honest answer: for the tabular fault data of Weeks 2–6, a tuned gradient-boosting model is often as good or better than a neural net.

Raw, Unstructured Data

Images from an inspection camera; long vibration waveforms. There are no hand-made features to feed a tree.

Learned Representations

Networks build their own features from raw input — that's Weeks 9–11 (CNNs, LSTMs, attention).

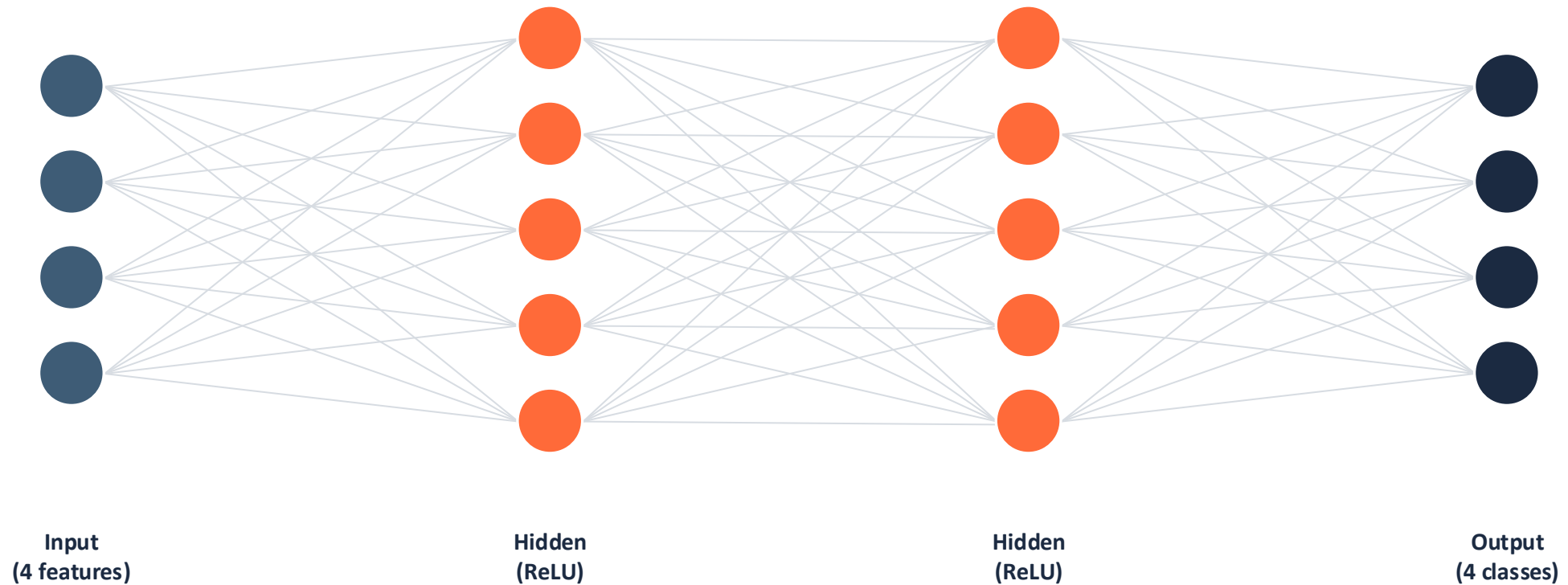
Control & RL

Policies that map states to actions, trained by interaction — impossible with `fit()/predict()` (Week 12).

Today is the tooling bridge — same problem as Week 2, new framework, so the framework is the only thing that's new.

From One Neuron to a Network

Week 1's artificial neuron, stacked into layers. Each circle is exactly the same weighted-sum-plus-activation unit you already saw.



Depth is what buys you nonlinearity: stacked linear layers alone collapse into one linear layer — the activation functions between them are what make it more than logistic regression.

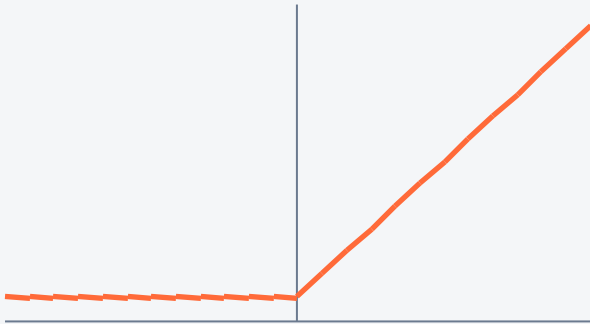
Activation Functions



The nonlinearity between layers. ReLU is the default for hidden layers — simple, fast, and it avoids the vanishing-gradient problem sigmoid has.

ReLU

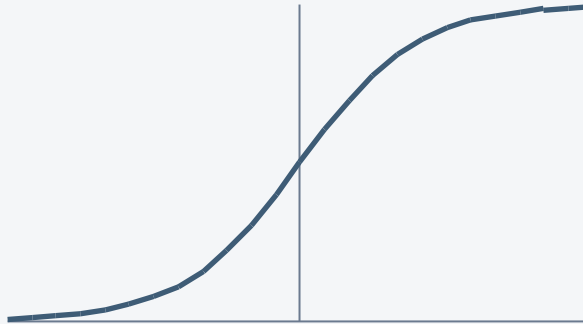
$\max(0, x)$



Default for hidden layers

Sigmoid

squashes to $(0, 1)$



Binary output layer

Tanh

squashes to $(-1, 1)$

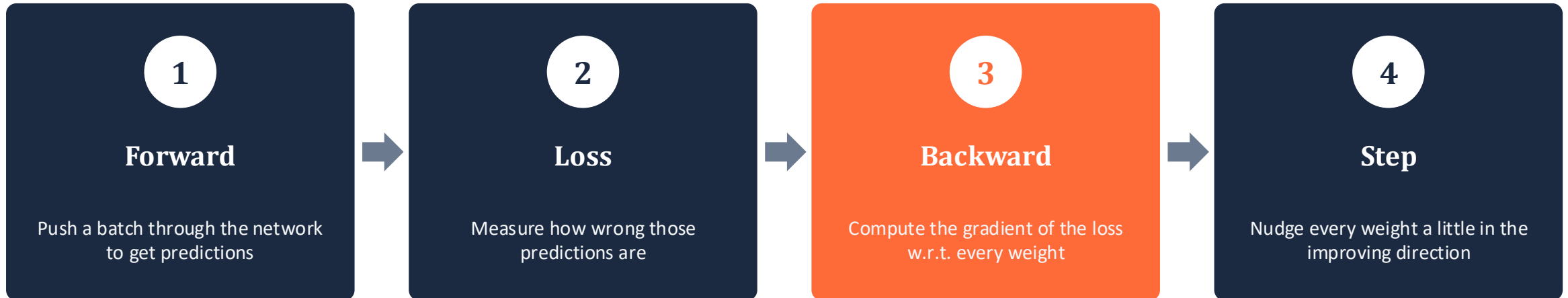


Zero-centered alternative

How a Network Learns: The Training Loop



scikit-learn hid this inside `.fit()`. In PyTorch you write it yourself — which is exactly why you'll finally see what “training” actually means.



↳ repeat for every batch, for every epoch

Backpropagation is just step 3: the chain rule applied backwards through the network, so every weight learns how much it contributed to the error.

PyTorch Tensors: NumPy Arrays That Track Gradients



If you know NumPy, you already know 90% of the tensor API. The two differences that matter: gradient tracking, and the ability to move to a GPU.

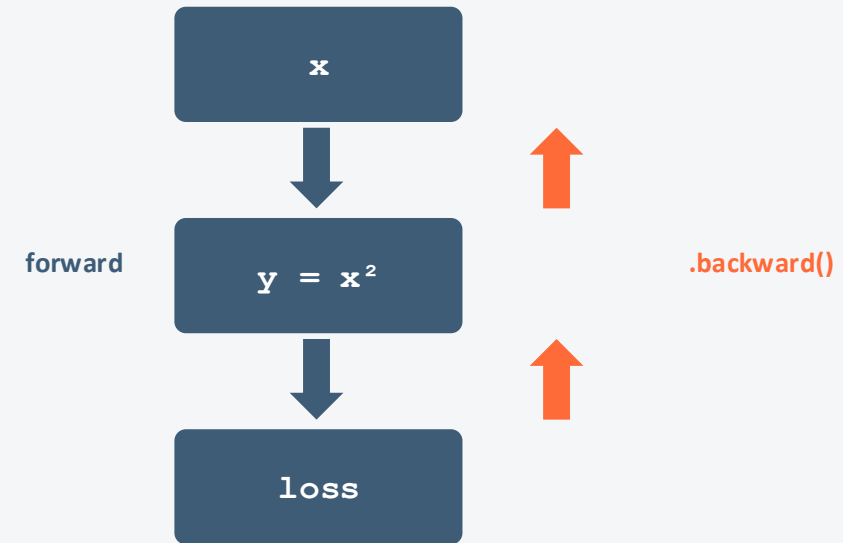
	NumPy	PyTorch
Create	<code>np.array([1., 2.])</code>	<code>torch.tensor([1., 2.])</code>
Zeros	<code>np.zeros((3, 4))</code>	<code>torch.zeros(3, 4)</code>
Matrix multiply	<code>a @ b</code>	<code>a @ b</code>
Shape	<code>a.shape</code>	<code>a.shape</code>
Track gradients	—	<code>requires_grad=True</code>
Move to GPU	—	<code>a.to("cuda")</code>

Autograd: PyTorch Does the Calculus



PyTorch records every operation you perform on a tensor. Call `.backward()`, and it walks that record in reverse to compute every gradient.

```
x = torch.tensor(3.0,  
                  requires_grad=True)  
  
y = x ** 2          #  $y = x^2$   
y.backward()        #  $dy/dx = 2x$   
  
print(x.grad)       # tensor(6.)
```



Defining a Model: nn.Module



Two ways to build the same MLP. Use Sequential for simple stacks; subclass Module when you need custom logic in the forward pass.

nn.Sequential

```
model = nn.Sequential(  
    nn.Linear(4, 32),  
    nn.ReLU(),  
    nn.Linear(32, 16),  
    nn.ReLU(),  
    nn.Linear(16, 4),  
)
```

Subclassing nn.Module

```
class FaultNet(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(...)  
  
    def forward(self, x):  
        return self.net(x)
```

Feeding the Model: Dataset & DataLoader



Neural nets train on mini-batches, not the whole dataset at once. DataLoader handles batching, shuffling, and iteration for you.

```
train_ds = TensorDataset(X_train_tensor, y_train_tensor)

train_dl = DataLoader(train_ds, batch_size=32, shuffle=True)
```

Why batches?

Full-dataset gradients are slow and memory-hungry; single samples are too noisy. Batches of 32–256 are the practical middle.

Why shuffle?

If your data is sorted by class, un-shuffled batches would each contain one class — the model would lurch around instead of converging.

The Training Loop, In Code

Five lines do the real work. Memorize this shape — every PyTorch project you write for the rest of the course reuses it.

```
for epoch in range(n_epochs):  
    for xb, yb in train_dl:  
        optimizer.zero_grad()      # clear last step's gradients  
        preds = model(xb)           # 1. forward  
        loss = loss_fn(preds, yb)   # 2. loss  
        loss.backward()             # 3. backward  
        optimizer.step()            # 4. update weights
```

The #1 beginner bug: forgetting `optimizer.zero_grad()`. PyTorch accumulates gradients by default, so without it, every step is polluted by the previous one.

Same Task, Two Frameworks



scikit-learn

- `model.fit(X, y)`
- Loop is hidden inside `.fit()`
- Great defaults, little control
- Weeks 1–6



PyTorch

- You write the loop yourself
- Full control of every step
- More code, far more flexibility
- Weeks 7–14

In the lab you'll run both on the same fault dataset — and find the accuracies land close together. That's the honest result.

Five Bugs You Will Hit This Week



Every one of these is normal. Recognizing the error message is most of the fix.

Forgot `zero_grad()`

Loss plateaus or diverges strangely — gradients are accumulating across steps.

Wrong dtype

“expected scalar type Long but found Float” — features must be float32, class labels int64.

Shape mismatch

“mat1 and mat2 shapes cannot be multiplied” — print `x.shape` between layers.

Softmax applied twice

CrossEntropyLoss expects raw logits. Don't add `nn.Softmax` to the output layer.

Forgot to scale features

Neural nets need scaled inputs just as much as Week 2's SVM did.

Today's Lab: Rebuild Week 2 in PyTorch

1

Tensors & autograd warm-up

Verify a gradient by hand before trusting the framework.

2

Prepare the fault data

Reuse the Week 2 pipeline, then convert to tensors.

3

Define an MLP & training loop

`nn.Module`, `CrossEntropyLoss`, `Adam` — written out in full.

4

Compare against scikit-learn

Same split, same metric. Which wins, and does it matter?

Key Takeaways



A neural network is Week 1's neuron, stacked — activations supply the nonlinearity.



Tensors are NumPy arrays that track gradients; autograd does the calculus.



forward → loss → backward → step. Every model in Weeks 8–14 uses this loop.

NEXT WEEK

PyTorch Mechanics

Custom modules, optimizers, saving models — and a regression project on robot joint torque.